Relevantz
Engineering. And Trust.

Code Quality

Developer Ground Rules for Writing Quality Code

Developer Ground Rules for Writing Quality Code

What defines “good code”?

There is no clear consensus on what good code actually means. Generally speaking, however, quality code is:

- Thoroughly tested and debugged — otherwise known as clean
- High-performance
- Sustainable and maintained
- Readable

Of course, different developers will offer different perspectives on this.

Developer Ground Rules for Writing Quality Code

Tips for writing code

1. Choose Industry-Specific Coding Standards.
2. Focus on Code readability.
3. Meaningful Names.
4. Avoid using a Single Identifier for multiple purposes.
5. Add Comments and Prioritize Documentation.
6. Efficient Data Processing.
7. Effective Version Control and Collaboration.
8. Effective Code Review and Refactoring.
9. Try to formalize Exception Handling.
10. Security and Privacy Considerations.
11. Standardize Headers for Different Modules.

Best Practices for Naming in C#

Best Practices for Naming in C#

There are following three terminologies are used to declare C# and .NET naming standards:

Camel Case (camelCase): In this standard, the first letter of the word always in small letter and after that each word starts with a capital letter.

Pascal Case (PascalCase): In this the first letter of every word is in capital letter.

Underscore Prefix (_underScore): For underscore (_), the word after _ use camelCase terminology.

Kind	Rule
Private field	_lowerCamelCase
Public field	UpperCamelCase
Protected field	UpperCamelCase
Internal field	UpperCamelCase
Property	UpperCamelCase
Method	UpperCamelCase
Class	UpperCamelCase
Interface	IUpperCamelCase
Local variable	lowerCamelCase
Parameter	lowerCamelCase

What is Clean Code?

What is Clean Code?

Clean code is a term used to refer to code that is easy to read, understand, and maintain.

It was made popular by **Robert Cecil Martin**, also known as Uncle Bob, who wrote "**Clean Code: A Handbook of Agile Software Craftsmanship**" in 2008. In this book, he presented a set of principles and best practices for writing clean code, such as using meaningful names, short functions, clear comments, and consistent formatting.

Ultimately, the goal of clean code is to create software that is not only functional but also readable, maintainable, and efficient throughout its life cycle

Why should we care about Clean Code?

Why should we care about Clean Code?

When teams adhere to clean code principles, the code base is easier to read and navigate, which makes it faster for developers to get up to speed and start contributing. Here are some reasons why clean code is essential:

- ✓ Readability and maintenance.
- ✓ Team collaboration.
- ✓ Debugging and issue resolution.
- ✓ Improved quality and reliability.

Characteristics of Clean Code

Characteristics of Clean Code

Avoid Hard-Coded Numbers

Use named constants instead of hard-coded values. Write constants with meaningful names that convey their purpose. This improves clarity and makes it easier to modify the code.

Before:

```
def calculate_discount(price):
   discount = price * 0.1                    //0% discount
   return price – discount
```

After :

```
def calculate_discount(price):
  TEN_PERCENT_DISCOUNT = 0.1
  discount = price * TEN_PERCENT_DISCOUNT
  return price - discount
```

Characteristics of Clean Code

Use Meaningful and Descriptive Names

Choose names for variables, functions, and classes that reflect their purpose and behavior. This makes the code self-documenting and easier to understand without extensive comments. As Robert Martin puts it, “A name should tell you why it exists, what it does, and how it is used. If a name requires a comment, then the name does not reveal its intent.”

Before:

```
def calculate_discount(price):
  TEN_PERCENT_DISCOUNT = 0.1
  discount = price * TEN_PERCENT_DISCOUNT
  return price – discount
```

After:

```
def calculate_discount(product_price):
   TEN_PERCENT_DISCOUNT = 0.1
   discount_amount = product_price *
TEN_PERCENT_DISCOUNT
   return product_price - discount_amount
```

Characteristics of Clean Code

Use Comments Sparingly, and When You Do, Make Them Meaningful

You don't need to comment on obvious things. Excessive or unclear comments can clutter the code base and become outdated, leading to confusion and a messy code base.

Before:

```
def group_users_by_id(user_id):
  # This function groups users by id
  # ... complex logic ...
  # ... more code …
```

After:

```
def group_users_by_id(user_id):
  """Groups users by id to a specific category (1-9).
  Args:
    user_id (str): The user id to be grouped.
  Returns:
    int: The category number (1-9) corresponding to
the user id.
  Raises:
    ValueError: If the user id is invalid or
unsupported.
  """
  # ... complex logic ...
  # ... more code …
```

Characteristics of Clean Code

Write Short Functions That Only Do One Thing

Follow the single responsibility principle (SRP), which means that a function should have one purpose and perform it effectively. Functions are more understandable, readable, and maintainable if they only have one job. It also makes testing them very easy.

If a function becomes too long or complex, consider breaking it into smaller, more manageable functions.

Before:

```
def process_data(data):
  # ... validate users...
  # ... calculate values ...
  # ... format output …
```

After:

```
def validate_user(data):
  # ... data validation logic ...
def calculate_values(data):
  # ... calculation logic based on validated data ...
def format_output(data):
  # ... format results for display …
```

Characteristics of Clean Code

Follow the DRY (Don't Repeat Yourself) Principle and Avoid Duplicating Code or Logic

Avoid writing the same code more than once. Instead, reuse your code using functions, classes, modules, libraries, or other abstractions. This makes your code more efficient, consistent, and maintainable. It also reduces the risk of errors and bugs as you only need to modify your code in one place if you need to change or update it.

Before:

```
def calculate_book_price(quantity, price):
  return quantity * price
def calculate_laptop_price(quantity, price):
  return quantity * price
```

After:

```
def calculate_product_price(product_quantity,
product_price):
 return product_quantity * product_price
```

Characteristics of Clean Code

Follow Established Code-Writing Standards

Know your programming language's conventions in terms of spacing, comments, and naming. Most programming languages have community-accepted coding standards and style guides.

Characteristics of Clean Code

Encapsulate Nested Conditionals into Functions

One way to improve the readability and clarity of functions is to encapsulate nested if/else statements into other functions.

Before:

```
def calculate_product_discount(product_price):
 discount_amount = 0
 if product_price > 100:
   discount_amount = product_price * 0.1
 elif price > 50:
   discount_amount = product_price * 0.05
 else:
   discount_amount = 0
 final_product_price = product_price - discount_amount
 return final_product_price
```

After:

```
def calculate_discount(product_price):
  discount_rate = get_discount_rate(product_price)
  discount_amount = product_price * discount_rate
  final_product_price = product_price -
discount_amount
   return final_product_price

def get_discount_rate(product_price):
 if product_price > 100:
   return 0.1
 elif product_price > 50:
   return 0.05
 else:
   return 0
```

Characteristics of Clean Code

Refactor Continuously

Regularly review and refactor your code to improve its structure, readability, and maintainability. Consider the readability of your code for the next person who will work on it, and always leave the code base cleaner than you found it.

Use Version Control

Version control systems meticulously track every change made to your code base, enabling you to understand the evolution of your code and revert to previous versions if needed. This creates a safety net for code refactoring and prevents accidental deletions or overwrites.

Use version control systems like GitHub, Git Lab, and Bitbucket to track changes to your code base and collaborate effectively with others.

C# Coding Guidelines

C# Coding Guidelines

Class Names

Use PascalCasing for class names and method names.

Why: consistent with the Microsoft's .NET Core, and easy to read.

```
public class ClientActivity
{
    public void ClearStatistics()
    {
        //...
    }
    public void CalculateStatistics()
    {
        //...
    }
}
```

Variable Names

Use camelCasing for local variables and method arguments.

Why: consistent with the Microsoft's .NET Core, and easy to read.

```
public class UserLog
{
    public void Add(LogEvent logEvent)
    {
        int itemCount = logEvent.Items.Count;
        // ...
    }
}
```

C# Coding Guidelines

Identifiers

Do not use Hungarian notation or any other type identification in identifiers.

Why: consistent with the Microsoft's .NET Core. In addition, the Visual Studio IDE makes it very easy to determine the type of a variable (via tooltips). It is best to avoid type indicators in identifiers.

```
// Correct
int counter;
string name;

// Avoid
int iCounter;
string strName;
```

Constants

Do not use Screaming Caps for constants or readonly variables.

Why: consistent with the Microsoft's .NET Core. Caps grap too much attention.

```
// Correct
public static const string ShippingType = "DropShip";

// Avoid
public static const string SHIPPINGTYPE = "DropShip";
```

C# Coding Guidelines

Abbreviations

Avoid using Abbreviations.

Exceptions: abbreviations commonly used as names, such as Id, Xml, Ftp, Uri

Why: consistent with the Microsoft's .NET Core. It prevents inconsistent abbreviations by different developers.

```
// Correct
UserGroup userGroup;
Assignment employeeAssignment;

// Avoid
UserGroup usrGrp;
Assignment empAssignment;

// Exceptions
CustomerId customerId;
XmlDocument xmlDocument;
FtpHelper ftpHelper;
UriPart uriPart;
```

Abbreviation Casing

Use PascalCasing for abbreviations 3 characters or more (2 chars are both uppercase).

Why: consistent with the Microsoft's .NET Core. Caps would grap visually too much attention.

```
HtmlHelper htmlHelper;
FtpTransfer ftpTransfer;
UIControl uiControl;
```

C# Coding Guidelines

No Underscores

Do not use Underscores in identifiers.

Exception: you can prefix private static variables with an underscore.

Why: consistent with the Microsoft's .NET Core. It makes code more natural to read (without 'slur'). Also avoids underline stress, i.e. inability to see underline.

```
// Correct
public DateTime clientAppointment;
public TimeSpan timeLeft;

// Avoid
public DateTime client_Appointment;
public TimeSpan time_Left;

// Exception
private DateTime _registrationDate;
```

Type Names

Use predefined type names instead of system type names like Int16, Single, UInt64, etc..

Why: consistent with the Microsoft's .NET Core. It makes code more natural to read.

```
// Correct
string firstName;
int lastIndex;
bool isSaved;

// Avoid
String firstName;
Int32 lastIndex;
Boolean isSaved;
```

C# Coding Guidelines

Implicit Types

Use implicit type var for local variable declarations.

Exception: primitive types (int, string, double, etc) use predefined names.

Why: removes clutter, particularly with complex generic types. Type is easily detected with Visual Studio tool tips

```
var stream = File.Create(path);
var customers = new Dictionary();

// Exceptions
int index = 100;
string timeSheet;
bool isCompleted;
```

Noun Class Names

Use noun or noun phrases to name a class.

Why: consistent with the Microsoft's .NET Core. Makes classes easy to remember.

```
public class Employee
{
}
public class BusinessLocation
{
}
public class DocumentCollection
{
}
```

C# Coding Guidelines

Interfaces

Use prefix interfaces with the letter I. Interface names are noun (phrases) or adjectives.

Why: consistent with the Microsoft's .NET Core.

```
public interface IShape
{
}
public interface IShapeCollection
{
}
public interface IGroupable
{
}
```

File Names

Name source files according to their main classes. Exception: file names with partial classes reflect their source or purpose, e.g. designer, generated, etc.

```
// Located in Task.cs
public partial class Task
{
    //...
}
```

```
// Located in Task.generated.cs
public partial class Task
{
    //...
}
```

C# Coding Guidelines

Namespaces

Organize namespaces with a clearly defined structure.

Why: consistent with the Microsoft's .NET Core. Maintains good organization of your code base.

```
// Examples
namespace Company.Product.Module.SubModule
namespace Product.Module.Component
namespace Product.Layer.Module.Group
```

Curly Brackets

Kindly vertically align curly brackets.

Why: Microsoft has a different standard, but developers have overwhelmingly preferred vertically aligned brackets.

```
// Correct
class Program
{
    static void Main(string[] args)
    {
    }
}
```

Source File Structure

Source File Structure

General Structure of a C# Program

C# programs consist of one or more files. Each file contains zero or more namespaces. A namespace contains types such as classes, structs, interfaces, enumerations, and delegates, or other namespaces. The following example is the skeleton of a C# program that contains all of these elements.

You can also create a static method named Main as the program's entry point, as shown in the following example:

```
// A skeleton of a C# program
using System;
namespace YourNamespace
{
   class YourClass
   {
   }

   struct YourStruct
   {
   }

   interface IYourInterface
   {
   }

   delegate int YourDelegate();

   enum YourEnum
   {
   }

   namespace YourNestedNamespace
   {
      struct YourStruct
      {
      }
   }

   class Program
   {
      static void Main(string[] args)
      {
         Console.WriteLine("Hello world!");
      }
   }
}
```

White Spaces

White Spaces

What are White Spaces?

In C#, "white-space" refers to characters that are used to improve code readability but are generally ignored by the compiler when parsing the code. These characters include:

- ✓ Spaces: ` `
- ✓ Tabs: \t
- ✓ Newlines: \n (line feed), \r (carriage return), and \r\n (Windows newline)

White Spaces

Purpose of White space:

Readability:

Whitespace helps organize code visually, making it easier for human programmers to understand and navigate. For example, indenting code blocks with spaces or tabs clearly defines their scope.

Separation:

Whitespace separates different elements of the code, such as keywords, identifiers, operators, and literals, preventing ambiguity.

White Spaces

Where Whitespace Matters and Doesn't:

- ✗ Ignored by Compiler (mostly).
- ✓ Significant within String Literals.
- ✓ Method Parameters.

Indentation

Indentation

What are Indentation?

Indentation in C# programming refers to the use of whitespace (spaces or tabs) at the beginning of lines of code to visually represent the hierarchical structure and scope of code blocks. While C# does not enforce indentation for code compilation (unlike languages like Python), it is a crucial element of good coding practice for enhancing readability and maintainability.

Indentation

Key aspects of indentation in C#:

- ✓ Readability.
- ✓ Common Convention: The widely accepted convention in C# is to use four spaces for each level of indentation, rather than tabs.
- ✓ Block Structure.
- ✓ IDE Support.
- ✓ EditorConfig: For team-based projects, .editorconfig files can be used to define consistent coding styles, including indentation rules.

Method Parameters

Method Parameters

What are Method Parameters?

In C#, method parameters are variables declared in a method's signature that define the type and name of the data a method can accept as input when it is called.

They allow methods to be more flexible and reusable by enabling them to operate on different data values without needing to be rewritten for each specific case.

Method Parameters

Defining a Method Parameters:

Parameters are listed within the parentheses after the method name in its declaration. Each parameter consists of a data type followed by a parameter name. Multiple parameters are separated by commas.

```
public void MyMethod(int number, string text)
{
    // Method logic using 'number' and 'text'
}
```

Method Parameters

Passing Arguments:

When a method with parameters is called, concrete values, known as arguments, are provided for each parameter. The arguments must be compatible with the declared parameter types.

```
MyMethod(10, "Hello"); // Calling MyMethod with arguments
```

Method Parameters

Types of Method Parameters in C#:

- ✓ Value Parameters.
- ✓ Reference Parameters (ref): Used to pass a variable by reference, meaning the method directly works with the original variable in memory.
- ✓ Output Parameters (out): Similar to ref parameters, but used when a method needs to return multiple values.
- ✓ Parameter Arrays (params): Allows a method to accept a variable number of arguments of a specific type.
- ✓ Named Parameters: Introduced in C# 4.0, these allow arguments to be passed by specifying the parameter name, rather than relying on their positional order.
- ✓ Optional Parameters (Default Parameters): Also introduced in C# 4.0, these allow parameters to have a default value, making them optional when calling the method.

Hard Coding

Hard Coding

What is Hard Coding?

Hard coding in a C# program refers to the practice of embedding specific values, configurations, or logic directly into the source code rather than obtaining them from external sources or allowing them to be dynamically determined at runtime.

Hard Coding

Examples of Hard Coding in C#:

- ✓ Literal Values: Directly assigning numerical or string literals to variables or using them in expressions.

```
int maxAttempts = 3; // Hard-coded value for maximum login attempts
string defaultMessage = "Welcome to the application!"; // Hard-coded string
```

- ✓ File Paths/Connection Strings: Embedding file paths or database connection strings directly in the code.

```
string filePath = "C:\\Data\\config.txt"; // Hard-coded file path
string connectionString = "Server=localhost;Database=MyDb;Integrated Security=True;"; // Hard-coded connection string
```

- ✓ Business Logic Parameters: Using fixed values within calculations or decision-making logic that might change over time.

Hard Coding

Implications of Hard Coding:

- ✓ Reduced Flexibility: Changes to hard-coded values require modification and recompilation of the source code, making updates cumbersome.
- ✓ Maintainability Issues: Locating and updating hard-coded values across a large codebase can be challenging and error-prone.
- ✓ Security Risks: Hard-coding sensitive information like passwords or API keys can expose them if the code is compromised.
- ✓ Limited Reusability: Code with hard-coded values is less adaptable to different environments or scenarios.

Code Comments

Code Comments

What are Code Comments?

Comments in C# programs are non-executable lines of text used to explain code, improve readability, and temporarily disable code sections. The C# compiler ignores comments during compilation.

Code Comments

There are three main types of comments in C#:

Single-Line Comments:
Start with two forward slashes (//).
Any text following // on the same line is considered a comment and ignored by the compiler.

Example:

```
int x = 10; // Declare an integer variable x and initialize it to 10
```

Code Comments

There are three main types of comments in C#:

Multi-Line Comments:
Start with /* and end with */.
All text between /* and */ across multiple lines is treated as a comment.

Example:

```
/*
This is a multi-line comment.
It can span multiple lines to provide detailed explanations
or to comment out a block of code.
*/
```

Code Comments

There are three main types of comments in C#:

XML Documentation Comments:
Start with three forward slashes (///).
They typically contain specific XML tags like <summary>, <param>, <returns>, etc., to describe the purpose of classes, methods, and other members.

Example:

```
 /// <summary>
/// This method calculates the sum of two integers.
/// </summary>
/// <param name="a">The first integer.</param>
/// <param name="b">The second integer.</param>
/// <returns>The sum of a and b.</returns>
public int Add(int a, int b)
{
    return a + b;
}
```

SonarQube

SonarQube

What is SonarQube?

SonarQube is an open-source platform for continuous inspection of code quality and security.

It helps developers identify and fix bugs, code smells, vulnerabilities, and code duplications by performing static code analysis.

Essentially, it acts as a code quality gatekeeper, helping teams deliver better, safer software.

SonarQube

Sonar Scanner

A sonar scanner, in the context of software development, is a tool used for static code analysis.

It examines source code to identify potential issues like bugs, vulnerabilities, and code smells without actually executing the code.

Specifically, the SonarScanner is a command-line tool that works in conjunction with SonarQube or SonarCloud to analyze code and report findings to the platform.

SonarQube

Sonar Source Code

Sonar Source Code refers to the code that SonarQube analyzes to assess code quality and security.

SonarQube, a SonarSource product, uses static analysis to detect bugs, code smells, and security vulnerabilities in source code written in various programming languages.

It integrates with CI/CD pipelines and IDEs to provide continuous feedback on code quality throughout the development lifecycle.

By using SonarQube, development teams can reduce technical debt, improve code maintainability, enhance security, and ultimately deliver higher quality software.

SonarQube

Sonar Analyzer

A Sonar analyzer, often referring to the tools offered by SonarSource, is a static code analysis tool used to improve code quality and security.

It identifies potential bugs, vulnerabilities, and code smells in various programming languages, helping developers write better code.

Sonar analyzers integrate with IDEs and CI/CD pipelines for real-time feedback and continuous monitoring.

SonarQube

SonarQube Database

SonarQube utilizes a relational database to store and manage the data collected during code analysis. This database serves as the central repository for all the information SonarQube gathers about your projects, including:

- ✓ Code analysis results.
- ✓ Project configuration.
- ✓ User and permission management.
- ✓ Historical data.

SonarQube

How to Use SonarQube?

- ✓ Install SonarQube.
- ✓ Access SonarQube.
- ✓ Create a Project in SonarQube.
- ✓ Generate a Token.
- ✓ Prepare your Project.
- ✓ Install SonarScanner.
- ✓ Configure SonarScanner.
- ✓ Execute the Scan.
- ✓ Access Project Dashboard.
- ✓ Explore Issues.

Code Review

Code Review

Code Review Principles

- ✓ Focus on Design and Correctness.
- ✓ Clarity and Readability.
- ✓ Test Coverage and Quality.
- ✓ Adherence to Standards and Best Practices.
- ✓ Security and Performance.
- ✓ Constructive and Respectful Feedback.
- ✓ Knowledge Sharing and Learning.
- ✓ Scope and Efficiency.
- ✓ Documentation and Build Integrity.
- ✓ Testability and Debuggability.

Code Review

Code Performance

- ✓ Algorithmic Complexity.
- ✓ Resource Utilization.
- ✓ Execution Speed.
- ✓ Scalability.
- ✓ Reliability.
- ✓ Efficient Data Structures and Algorithms.
- ✓ Resource Management.
- ✓ Compiler and Runtime Optimizations.

Error and Exceptions

Error and Exceptions

Errors

- ✓ Errors are typically severe, unrecoverable problems that occur at the system level or due to critical resource limitations.
- ✓ They often indicate issues that are beyond the program's control and cannot be handled gracefully by application code.
- ✓ Examples include OutOfMemoryError (when the Java Virtual Machine runs out of memory) or StackOverflowError (when the call stack overflows).
- ✓ Errors generally lead to the abnormal termination of the program.

Error and Exceptions

Exceptions

- ✓ Exceptions are less severe issues that occur during the normal flow of program execution due to exceptional conditions or unexpected events.
- ✓ They represent situations that can often be anticipated and handled by the program using constructs like try-catch blocks.
- ✓ Examples include FileNotFoundException (when a program tries to access a non-existent file), NullPointerException (when trying to access a member of a null object), or IOException (during input/output operations).
- ✓ Exceptions allow for controlled recovery and enable the program to continue execution or gracefully exit, depending on the handling strategy.

Code Reusability

Code Reusability

Code reusability refers to the practice of designing and writing software components, modules, or libraries in a way that allows them to be used in multiple contexts or projects with minimal or no modification. Key aspects and benefits of code reusability include:

- ✓ Reduced Development Time and Cost.
- ✓ Improved Code Quality and Reliability.
- ✓ Enhanced Maintainability.
- ✓ Increased Consistency and Standardization.
- ✓ Facilitates Modular Design.
- ✓ Promotes Collaboration.
- ✓ Supports Scalability.

Thread Safe Coding

Thread Safe Coding

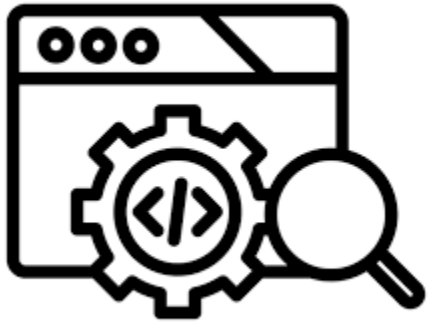
Thread-safe coding ensures that a program behaves correctly even when multiple threads access and modify shared data concurrently. Key points for achieving thread safety include:

- ✓ Synchronization Mechanisms.
- ✓ Immutable Objects.
- ✓ Atomic Operations.
- ✓ Thread-Local Storage.

Code Coverage

Code Coverage

- ✓ Code coverage in software testing is a metric that indicates the extent to which the source code of a program is executed when a particular test suite runs.
- ✓ It helps identify untested parts of the codebase, ensuring thorough testing and reducing the likelihood of bugs.
- ✓ Higher code coverage generally suggests better testing and a lower risk of undetected issues.



Code Coverage

Types of Code Coverage:

- ✓ Line Coverage: Measures the percentage of executable lines of code that have been executed.
- ✓ Function Coverage: Measures the percentage of functions that have been called at least once.
- ✓ Branch Coverage/Decision Coverage: Measures the percentage of conditional branches (e.g., if statements) that have been executed with both true and false outcomes.
- ✓ Statement Coverage: Measures the percentage of executable statements that have been executed.
- ✓ Condition Coverage: Measures the percentage of boolean sub-expressions within conditional statements that have been evaluated to both true and false.
- ✓ Path Coverage: Measures the percentage of possible execution paths through the code that have been executed.

Ncover and NCrunch

NCover

- ✓ NCover is a .NET code coverage tool that analyzes your application's code to identify which parts have been tested and which have been missed.
- ✓ It provides detailed metrics like sequence point, branch, and method coverage to help developers improve testing and code quality.
- ✓ By visualizing coverage data, NCover helps pinpoint areas needing more testing, ultimately leading to more robust and reliable applications.

NCrunch

- ✓ NCrunch is a continuous testing tool for .NET developers that integrates with Visual Studio.
- ✓ It provides automatic, parallel test execution, code coverage analysis, and runtime data inspection, improving developer productivity and code quality.
- ✓ NCrunch is a commercial product with a focus on providing a seamless and efficient testing experience within the development workflow.

THANK YOU

Nisanth Saravanan

About Relevantz

Relevantz Technology Services Inc. has been delivering relevant technology solutions for leading enterprises and ISVs for over 25+ years. Our team of 1200+ software engineers across 4 global offices serves customers across the finance, healthcare, insurance, media, telecom, retail, and technology sectors. Learn more at www.relevantz.com or @relevantztech.